Part 1: Architectural Scenarios

Saga Pattern - Contextual Application

Let’s define some microservices involved in customer onboarding:

- Customer Service
- Account Service
- Inventory Service
- Customer Wallet Service
- Warehouse Service

Choreography

In choreography, each service communicates with other services as needed, without any central mediator.

In the context of customer onboarding:

1. The first request reaches the Customer Service.
2. Customer Service calls the Account Service.
3. Customer Service calls the Inventory Service.
4. Account Service calls the Wallet Service.
5. Inventory Service calls the Warehouse Service.

In this approach, services coordinate the workflow among themselves.

Orchestration

In orchestration, a central mediator service coordinates the entire workflow.

In the context of customer onboarding:

1. The first request reaches the Customer Onboarding Mediator.
2. The mediator service calls the Account Service.
3. The mediator service calls the Inventory Service.
4. The mediator service calls the Wallet Service.

5. The mediator service calls the Warehouse Service.

In this approach, the mediator controls the execution flow and coordination between services.

My Choice: Orchestration

I would strongly advocate for the Orchestration pattern in this scenario because the onboarding process involves 8–10 microservices, making the workflow fairly complex.

With orchestration:

- We can clearly control which services are mandatory and which are optional.
- Error handling and rollback management become easier.
- The workflow remains centralized and easier to maintain.

Why Not Choreography?

The primary weakness of choreography in this context is the complexity of error handling and distributed transaction management.

As the number of services increases, multiple services start behaving like mediators. In this example:

- Customer Service
- Account Service
- Inventory Service

all begin coordinating parts of the workflow, which increases coupling and makes the system harder to understand and maintain.

Guaranteed Once-Only Processing

Cases Regarding PGW

The following cases may occur when communicating with the Payment Gateway (PGW):

1. The call reaches the PGW and successfully returns a result — **Success Case**
2. The call does not reach the PGW and no payment is processed — **Failure Case**
3. The call reaches the PGW, but the response is lost — **Failure Case**

Workflow

Default Strategy

Our goal is to guarantee exactly-once processing.

First, the system will generate a unique identifier for each payment request, such as a custom ID or order ID.

A pessimistic locking mechanism will be implemented. Any request to the payment service must acquire a lock based on the unique identifier.

The system will maintain the following payment states:

- Processing
- Success
- Fail
- Timeout/Unknown

The lock will only be released after the current owner updates the request with one of the final states.

- In case of failure, retries will be performed based on the error status and business policy.
- In case of timeout or unknown status, retries will depend on the availability of the query API and business policy.

Strategy A — Relational Database

In this strategy, the database will act as the primary source for both lock management and status tracking.

Primary Mode of Failure

The database becomes the primary point of failure.

A special failure scenario occurs when the payment service successfully processes the payment, but the success state is not committed to the database.

In such cases, a lock timeout mechanism can resolve the issue by marking the transaction status as `Timeout/Unknown`.

Strategy B — Distributed Redis Cache

In this strategy, a distributed Redis cache will be used for lock management, as Redis provides strong support for distributed locking mechanisms. And db locking will be used as fallback.

For status management:

- Redis will be used as a write-through cache.
- Redis lock time out will be greater than PGW call timeout.
- The database will remain the primary source of truth.

In the event of a cache miss, the database will be consulted.

If a cache failure or disaster occurs, the payment service will either:

- Halt processing, or
- Fallback to the database-based locking mechanism.

Primary Mode of Failure

The primary mode of failure remains the database, since it is still the source of truth for transaction status management.

Resolving Conflicting Knowledge

Source of Truth

A single source of truth will be maintained for explicit policy-related semantic memory using a relational database.

Metadata

The following metadata fields will be maintained:

- `Return_policy_deadline`
- `Last_updated_date`
- `Last_updated_document_id`
- `Updated_document_type`
- `Effective_from`
- `Effective_to`
- `IsPromotional`
- `Retrieval_confidence`
- `User_approval`

Retrieval Method

The prompt strategy will be simple and explicit.

The LLM will be instructed to retrieve return policy information directly from the database using SQL queries.

Prompt Strategy

1. Fetch all metadata from the database based on the `Effective_from` and `Effective_to` dates.
2. Filter the data using:
    - Confidence score thresholds
    - User approval status
3. Promotional policies will override default policies only when the metadata constraints match the user context.
4. If no valid data exists, the LLM must not invent any policy. Instead, the request should be escalated to a human.

Insert / Update Method

Whenever a new document is ingested, the LLM will extract the updated return policy and store it in the database accordingly.

Data Extraction Strategy

Structured Documents

If the document is structured, the LLM will receive explicit instructions regarding which sections or fields to analyze.

Unstructured Documents

If the document is unstructured, the LLM will use reasoning to determine whether a new return policy has been introduced.

In both cases:

- The LLM must cite the source documents.
- The LLM must assign confidence scores to the extracted data.
- Any extraction below the confidence threshold will be escalated for human review.